Reverse-Engineering — Respostas

To-Do App Kotlin Multiplatform · Respostas baseadas na inspeção do código-fonte do projeto

Alunos: Paulo Fernandes e Jean Lucas
Matéria: Desenvolvimento Mobile

1. Estrutura do Projeto

PROJECT STRUCTURE

What are the main parts of the project, and where can you find:

- UI/screens;
- data models;
- SQLite/database code;
- navigation;
- notification code?

Briefly describe how the project is organized.

O projeto é um app **Kotlin Multiplatform (KMP) com Compose Multiplatform**, organizado em torno de um módulo único `composeApp` com fontes separadas por plataforma:

- **UI/telas**: `composeApp/src/commonMain/kotlin/br/edu/ifpe/todo/ui/`
- `ui/App.kt` — raiz da UI e máquina de estados de navegação;
- `ui/tasklist/TaskListScreen.kt` — lista de tarefas com filtros;
- `ui/taskeditor/TaskEditorScreen.kt` — criação/edição de tarefa;
- `ui/categories/CategoriesScreen.kt` — gestão de categorias;
- `ui/components/SharedComponents.kt` — componentes reutilizáveis (`EmptyState`, `CategoryPill`, etc.).
- **Modelos de dados**: `commonMain/.../domain/Models.kt` — `Task`, `Category`, enum `TaskStatusFilter`.
- **SQLite/banco**: `commonMain/sqldelight/br/edu/ifpe/todo/data/db/Todo.sq` (schema + queries SQLDelight), `data/db/DatabaseDriverFactory.kt` (interface), com implementações em `androidMain/.../database/AndroidDatabaseDriverFactory.kt` e `iosMain/.../database/IosDatabaseDriverFactory.kt`. A camada de acesso é `data/TodoRepository.kt`.
- **Navegação**: `commonMain/.../ui/App.kt` — `sealed interface Screen` + `mutableStateOf<Screen>` (não usa biblioteca de navegação).
- **Notificações**: `commonMain/.../notifications/ReminderScheduler.kt` (`expect class`) e `ReminderCoordinator.kt` (regras de negócio); implementações em `androidMain/.../notifications/` (`ReminderScheduler.android.kt`, `ReminderReceiver.kt`, `NotificationChannel.kt`) e `iosMain/.../notifications/ReminderScheduler.ios.kt`.
- **Infra compartilhada**: `AppContainer.kt` (container de DI simples), `util/TaskDueUtils.kt` e `util/DateFormatters.kt` (datas), entry points em `MainActivity.kt` (Android) e `MainViewController.kt` (iOS).

Organização: código compartilhado vive em `commonMain`; tudo que é específico de plataforma (driver SQLite, notificações, formatação de data) fica em `androidMain` / `iosMain` via `expect` / `actual` ou interfaces.

---

2. Arquitetura e Estado

ARCHITECTURE AND STATE

How is application state managed?

Explain how the UI is updated after an operation such as:

- creating a task;
- editing a task;
- marking a task as completed.

Does the project use any recognizable architectural pattern or state-management approach?

O padrão é **MVVM (Model–View–ViewModel)** com `StateFlow`:

- Cada tela tem um ViewModel (`TaskListViewModel`, `TaskEditorViewModel`, `CategoriesViewModel`) que expõe um `StateFlow<UiState>`.
- As telas leem o estado com `collectAsStateWithLifecycle()` — ex.: `TaskListScreen.kt:66`, `TaskEditorScreen.kt:69`.
- Os ViewModels são criados com `viewModel(factory = viewModelFactory { initializer { ... } })`, injetando `AppContainer.repository`.

Atualização da UI após operações:

- **Criar/editar tarefa**: `TaskEditorViewModel.save()` (`TaskEditorViewModel.kt:110`) grava no repositório. O repositório (`TodoRepository.kt`) expõe Flows reativos do SQLDelight (`observeTasks()` via `.asFlow().mapToList()`), que emitem automaticamente quando a tabela `task` muda. O `TaskListViewModel.uiState` (`TaskListViewModel.kt:32`) combina esses Flows, então a lista se atualiza sozinha — não há "refresh manual".
- **Marcar como concluída**: `TaskListViewModel.toggleCompleted()` (`TaskListViewModel.kt:70`) chama `repository.setTaskCompleted(...)`; o mesmo Flow reativo dispara a recomposição da lista.
- **Filtros**: `statusFilter` e `categoryFilterId` são `MutableStateFlow`s combinados ao Flow de tarefas; a filtragem é feita em memória (`filteredTasks`, `TaskListViewModel.kt:50`).

Abordagem reconhecível: **MVVM + Fluxo unidirecional de dados (UDF)** — ViewModel emite `UiState` imutável, View renderiza, eventos voltam por chamadas de funções do ViewModel, que alteram o estado (diretamente ou via banco).

---

3. Persistência com SQLite

SQLITE PERSISTENCE

How is SQLite used in the application?

Identify:

- where the database is created;
- how tasks and categories are stored;
- where create, read, update, and delete operations are implemented.

- **Criação do banco**: via **SQLDelight**. O schema é `Todo.sq` (`commonMain/sqldelight/.../Todo.sq`), que gera a classe `TodoDatabase` (configurada em `composeApp/build.gradle.kts:91-96`). O driver é criado por plataforma:
- Android: `AndroidDatabaseDriverFactory.createDriver()` → `AndroidSqliteDriver(..., name = "todo.db")` (`AndroidDatabaseDriverFactory.kt:10-15`);
- iOS: `IosDatabaseDriverFactory.createDriver()` → `NativeSqliteDriver(..., "todo.db")`.
- Inicialização: `AppContainer.init(driver)` (`AppContainer.kt:17`), chamada a partir de `initializeApp(context)` em `AndroidAppInit.kt` (via `MainActivity.onCreate`) e de `MainViewController()` no iOS.
- **Armazenamento**: duas tabelas em `Todo.sq`:
- `category(id INTEGER PK AUTOINCREMENT, name TEXT NOT NULL)`;
- `task(id PK, title, description, completed INTEGER 0/1, due_at, created_at, category_id REFERENCES category(id) ON DELETE SET NULL)`.
- Datas são persistidas como epoch milissegundos (`due_at`, `created_at`).
- **Operações CRUD**: implementadas em `TodoRepository.kt` delegando às queries do SQLDelight:
- **Create**: `insertTask()` (linha 64), `insertCategory()` (linha 108);
- **Read**: `observeTasks()` (linha 32), `observeCategories()` (linha 50), `fetchTask()` (linha 55);
- **Update**: `updateTask()` (linha 82), `setTaskCompleted()` (linha 100), `renameCategory()` (linha 113);
- **Delete**: `deleteTask()` (linha 104), `deleteCategory()` (linha 117).

---

4. Seguindo uma Operação: Criar Nova Tarefa

FOLLOW ONE OPERATION

Trace what happens when the user creates a new task.

Start from pressing **Save** and follow the execution until:

1. the task is stored in SQLite;
2. the task appears in the task list;
3. a notification is scheduled, if a due date exists.

Describe the main functions/components involved.

Fluxo completo a partir do toque em **Salvar** (`TaskEditorScreen.kt:84`):

1. **Toque em "Salvar"** → `viewModel.save(onSaved)` é chamado (`TaskEditorScreen.kt:84` → `TaskEditorViewModel.kt:110`).
2. **Validação**: `save()` retorna imediatamente se `title` estiver em branco (o botão também fica desabilitado via `uiState.canSave`).
3. **Conversão da data**: `TaskDueUtils.combineDueMillis(dueDate, dueTime)` (`TaskDueUtils.kt:44`) combina data+hora em epoch millis (hora padrão 23:59 se só houver data).
4. **Persistência (SQLite)**: dentro de `viewModelScope.launch`, como `isNewTask == true`, chama `repository.insertTask(...)` (`TaskEditorViewModel.kt:120` → `TodoRepository.kt:64`), que executa a query `insertTask` do `Todo.sq` e recupera o id com

`selectLastInsertRowid()`.

5. **Busca da tarefa salva**: `repository.fetchTask(savedTaskId)` carrega a linha completa de volta (`TaskEditorViewModel.kt:127`).
6. **Notificação**: `reminders.onTaskChanged(task)` (`TaskEditorViewModel.kt:140` → `ReminderCoordinator.kt:20`). Se a tarefa não estiver concluída e `dueAtMillis` for futuro, chama `scheduler.ensurePermission { ... scheduler.schedule(task.id, task.title, task.dueAtMillis) }`. No Android, `ReminderScheduler.android.schedule()` agenda um `AlarmManager.setExactAndAllowWhileIdle` cujo `PendingIntent` dispara o `ReminderReceiver`, que publica a notificação; no iOS, `ReminderScheduler.ios.schedule()` cria um `UNNotificationRequest` com `UNCalendarNotificationTrigger`.
7. **Navegação de volta**: `onSaved()` é invocado → `App.kt` volta para `Screen.TaskList`.
8. **Aparece na lista**: a lista não é recarregada manualmente — o `insertTask` dispara o Flow do SQLDelight, `TaskListViewModel.uiState` emite novo valor e a `LazyColumn` de `TaskListScreen` recompoõe com a nova tarefa.

---

5. Navegação

> **NAVIGATION**
> How does navigation between screens work?
> In particular:
> - how does the app navigate from the task list to the task editor?
> - when editing a task, what information is passed between screens?
>
> For example: task ID, full object, shared state, or another approach.

A navegação é **baseada em estado**, sem biblioteca externa, toda dentro de `ui/App.kt`:

- Um `sealed interface Screen` (`App.kt:17`) define as três telas: `TaskList`, `TaskEditor(taskId: Long?)`, `Categories`.
- O estado atual é um `var current by remember { mutableStateOf<Screen>(...) }` (`App.kt:26`), com um `when` renderizando a tela correspondente.
- **Lista → editor**: o FAB chama `onCreateTask` → `current = Screen.TaskEditor(null)`; tocar num item chama `onOpenTask(task.id)` → `current = Screen.TaskEditor(taskId)` (`App.kt:30-31`).
- **Informação passada ao editar**: **apenas o `taskId: Long?`**. `taskId == null` significa criação; `taskId != null` faz o `TaskEditorViewModel.load()` buscar a tarefa completa via `repository.fetchTask(taskId)` (`TaskEditorViewModel.kt:56`). Não passa o objeto inteiro nem usa estado compartilhado entre telas.
- Callbacks (`onCancel`, `onSaved`, `onDeleted`, `onDone`) apenas mudam `current` de volta para `Screen.TaskList`.

---

6. Notificações

> **NOTIFICATIONS**
> How are task reminders implemented?
> Explain:
> - how a notification is scheduled;
> - how it is associated with a task;
> - what happens when the due date changes;
> - what happens when the task is completed or deleted.

Implementação em camadas:

- **Agendamento**: `ReminderCoordinator.onTaskChanged(task)` (`ReminderCoordinator.kt:20`) centraliza as regras:
- concluída, sem prazo ou prazo no passado → `scheduler.cancel(task.id)`;
- prazo futuro e pendente → `ensurePermission` e, se concedido, `scheduler.schedule(task.id, task.title, dueAtMillis)`.
- **Associação à tarefa**: não existe coluna `notificationId`. O identificador é **derivado do `task.id`**:
- Android: `requestCodeFor(taskId) = taskId.toInt()` no `PendingIntent` (`ReminderScheduler.android.kt:121`) e `notificationId = taskId.toInt()` no `ReminderReceiver.onReceive()`;
- iOS: identificador de string `"task_$taskId"` (`ReminderScheduler.ios.kt:81`).
- **Data alterada**: ao salvar a edição, `onTaskChanged` é chamado de novo; como `schedule` usa o mesmo id/requestCode, o novo agendamento **substitui** o anterior (`FLAG_UPDATE_CURRENT` no Android; no iOS um `UNNotificationRequest` com o mesmo identificador substitui o pendente).
- **Concluída ou reaberta**: concluir → `cancel` (via `onTaskChanged` com `completed=true`, chamado também por `TaskListViewModel.toggleCompleted`, linha 74); reabrir com prazo futuro → `schedule` novamente.

- **Excluída**: `TaskEditorViewModel.deleteTask()` / `TaskListViewModel.deleteTask()` chamam `reminders.onTaskDeleted(id)` → `ReminderCoordinator.onTaskDeleted` → `scheduler.cancel(taskId)` (remove alarme no Android; `removePendingNotificationRequests` no iOS).
- **Permissão negada**: nunca quebra o app — no Android a `POST_NOTIFICATIONS` negada só resulta em `SecurityException` capturado no envio (`ReminderReceiver.kt:38-40`); no iOS o callback de `requestAuthorization` simplesmente não agenda exibição.

---

7. Decisões do Agente (não especificadas na atribuição)

AGENT DECISIONS

Identify at least **two important decisions made by the coding agent that were not explicitly specified in the assignment**.

Examples:

- architecture;
- libraries;
- state-management strategy;
- navigation approach;
- SQLite abstraction;
- project structure.

For each one, explain what the agent chose.

1. **Persistência com SQLDelight em vez de Room ou SQLite cru**: a spec pedia apenas "SQLite local". O agente escolheu **SQLDelight 2.0.2**, que gera código tipado a partir de `Todo.sq` e funciona igual em Android e iOS, evitando duplicar a camada de DAO. Complementarmente, criou a interface `DatabaseDriverFactory` (em vez de `expect class`) para injetar o driver nativo de cada plataforma, com instanciação centralizada no `AppContainer` (DI manual minimalista).
2. **Navegação por estado (sealed class) em vez de biblioteca de navegação**: em vez de usar `navigation-compose` (que estava em versão instável/alfa no ciclo do CMP 1.6), o agente implementou uma máquina de estados de 3 telas com `mutableStateOf<Screen>` em `App.kt`, passando **somente o `taskId`** entre lista e editor. Decisão documentada no `BUILD_LOG.md` (Entrada 1) como consciente: app pequeno, transparente e testável.

Outras decisões relevantes: MVVM multiplataforma com `ViewModel`/`StateFlow` da JetBrains (não AndroidX); `expect/actual` para notificações (AlarmManager vs UNUserNotificationCenter); datas como epoch millis com `kotlinx-datetime`; políticas de UX não pedidas (hora padrão 23:59, `ON DELETE SET NULL` em categorias, id de notificação derivado do `task.id`).

---

8. Análise do BUILD_LOG

BUILD_LOG ANALYSIS

Using `BUILD_LOG.md`, identify:

- one problem or bug encountered during development;
- how the agent attempted to solve it;
- whether the first solution worked;
- what was eventually done.

Then answer:

What did the build log help you understand that would have been harder to discover by looking only at the final code?

- **Problema/bug**: na primeira compilação (`./gradlew :composeApp:assembleDebug`, Entrada 10), entre outros, o SQLDelight 2.x **não retornava o id gerado no INSERT** — inserts retornam `Unit` no SQLDelight 2.x (sem auto-`rowid`), e o código dependia do id para buscar a tarefa recém-criada e agendar a notificação com ele.
- **Tentativa de solução**: adicionar a query `selectLastInsertRowid: SELECT last_insert_rowid() AS id` no `Todo.sq` e, no `TodoRepository`, chamar `lastInsertRowId()` logo após cada INSERT (`insertTask`/`insertCategory`).
- **Funcionou à primeira?** Sim para esse problema específico — mas a Entrada 10 mostra que o build inicial falhou com **8 erros encadeados** que precisaram ser corrigidos na ordem em que o compilador os revelava (grupo Maven errado do SQLDelight `com.squareup` → `app.cash`, `kotlin.time.Instant` inexistente no Kotlin 2.0.21, assinatura de `mapToList` exigindo `CoroutineContext`, objeto de queries gerado ser `todoQueries`, ícone de notificação inexistente, `@OptIn` de APIs M3, `monthNumber` etc.). Só depois o `assembleDebug` passou.
- **O que foi feito no fim**: corrigidos os 8 problemas, APK gerado (~8,9 MB); depois, validação em emulador (Entrada 11, com limitação de KVM) e preparo para celular físico via Android Studio/ADB Wi-Fi (Entrada 12).

O que o BUILD_LOG ajudou a entender que seria difícil ver só no código final: o **processo** — que a arquitetura "limpa" atual é resultado de várias correções iterativas reveladas pelo compilador (ex.: a query artificial `selectLastInsertRowid` existe por uma limitação

da API do SQLDelight 2.x, não por design ideal), que decisões como SQLDelight e navegação-por-estado foram escolhas deliberadas com alternativas consideradas e descartadas, e limitações do ambiente (iOS não compilável no Linux, sem KVM no emulador) que não são visíveis inspecionando apenas o código.